

Práctica 4 - Ejercicios 1.4

Ejercicio 1

Analiza el siguiente código sin ejecutarlo y predice qué fecha representa cada variable.

Variable	Fecha que representa
fechaA	10 de enero de 2026
fechaB	1 de enero de 2027
fechaC	1 de enero de 1970 00:00:02.026
fechaD	28 de febrero de 2026
fechaE	28 de febrero de 2026 (formato no estándar, depende del motor)

Ejercicio 2

Función `calcularDiasDiferencia(fechaInicio, fechaFin)` que recibe dos cadenas de texto en formato `YYYY-MM-DD` y devuelve el número entero de días transcurridos entre ambas, usando la diferencia en milisegundos con `getTime()` y redondeando con `Math.round()`.

fechas.js

```
1 function calcularDiasDiferencia(fechaInicio, fechaFin) {
2   const inicio = new Date(fechaInicio);
3   const fin = new Date(fechaFin);
4
5   const diferenciaMs = fin.getTime() - inicio.getTime();
6
7   return Math.round(diferenciaMs / 86400000);
8 }
```

Salida por consola:

Consola

```
> 73
> 0
> -73
```

Ejercicio 3

Función `obtenerUltimoDiaMes(anio, mes)` que devuelve los días que tiene el mes indicado, pasándole el mes en formato humano (1 = enero, 2 = febrero...) y aprovechando el desbordamiento del día 0.

fechas.js

```
1 function obtenerUltimoDiaMes(anio, mes) {  
2     return new Date(anio, mes, 0).getDate();  
3 }
```

Salida por consola:

Consola

```
> 28  
> 29  
> 31
```

Ejercicio 4

Función `formatearFechaEspanola(fecha)` que recibe un objeto `Date` y devuelve la fecha con el formato exacto `DD/MM/YYYY HH:mm`, rellenando con ceros a la izquierda mediante `padStart(2, "0")`.

fechas.js

```
1 function formatearFechaEspanola(fecha) {  
2     const dia = String(fecha.getDate()).padStart(2, "0");  
3     const mes = String(fecha.getMonth() + 1).padStart(2, "0");  
4     const anio = String(fecha.getFullYear()).padStart(4, "0");  
5     const horas = String(fecha.getHours()).padStart(2, "0");  
6     const minutos = String(fecha.getMinutes()).padStart(2, "0");  
7  
8     return dia + "/" + mes + "/" + anio + " " + horas + ":" + minutos;  
9 }
```

Salida por consola:

Consola

```
> 28/09/2026 12:30  
> 05/01/2026 07:05
```

Ejercicio 5

	Java	JavaScript
Tipo	Lenguaje tradicional, compilado	Lenguaje de script, interpretado
Ejecución		

	Java	JavaScript
	Bytecode sobre la máquina virtual (JVM)	El motor del navegador lo interpreta línea a línea
Tipado	Fuertemente tipado	Débilmente tipado
Detección de errores	En tiempo de compilación	En tiempo de ejecución
Orientación	Orientado a objetos	Orientado a eventos
Base de datos	Acceso directo (JDBC)	Sin acceso directo, usa el DOM o un servidor

Ejercicio 6

- Es la única tecnología que ejecuta lógica en el navegador.
- No necesita compilación ni instalar nada: se abre la página y listo.
- Portabilidad total: el mismo código funciona en el navegador de cualquier ordenador, tableta o móvil.
- Agilidad: se guarda, se recarga la página y se ve el resultado.
- Permite modificar la página y pedir datos al servidor sin recargarla.
- Ecosistema maduro y gratuito (React, Vue, Angular, npm, TypeScript).
- También se ejecuta fuera del navegador: en consola con Node.js y como app con Electron.

Ejercicio 7

Script `calculo.js` que intenta hacer una operación matemática con la variable `resultado`, que no ha sido declarada previamente.

```

calculo.js
1  const numero1 = 15;
2  const numero2 = 25;
3
4  const suma = numero1 + numero2;
5  console.log("numero1 + numero2 = " + suma);
6
7  const total = resultado * numero1;
8  console.log("resultado * numero1 = " + total);
9
10 console.log("El script termina aqui.");

```

Salida por consola:

Consola

> numero1 + numero2 = 40

× Uncaught ReferenceError: resultado is not defined calculo.js:7:15

El intérprete ejecuta las líneas 1 a 5 con normalidad, se detiene en la línea 7 con el `ReferenceError` y la línea 10 nunca llega a mostrarse.